

PROJETO.....**Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas****INTEGRANTES DO PROJETO e RA'S**

Caio Alves de Oliveira	-	26029746
Guilherme Santiago de Aguiar	-	17021144
Matheus Barbosa Tomaz Fraga	-	26029872
Pedro Paulo Pereira Celebroni	-	26029511

São Paulo

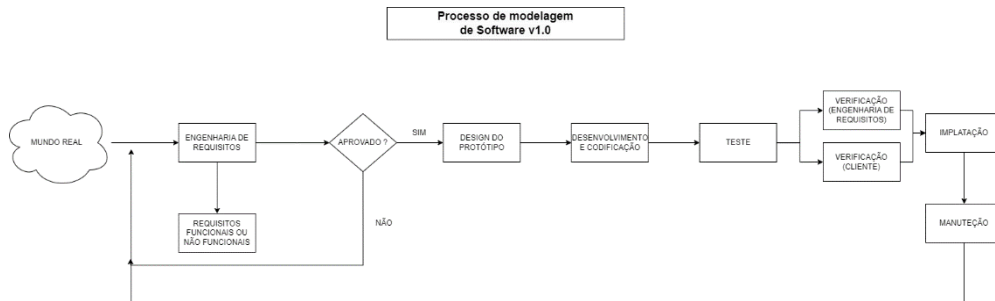
2026

Sumário

1 INTRODUÇÃO	3
1.1 – Processo de desenvolvimento de Software	3
1.2 Apresentação	3
2. DESIGN SPRINT – Ideação e prototipação do desafio.....	5
2.1 Desafio.....	5
2.2 Entender Mapear	5
2.2.1 – Entendimento e observação. COMO PODEMOS?	5
2.2.2 – Pesquisa Desk	5
2.3 Ideação – desenho da solução (trilha do usuário)	6
2.4 Prototipagem.....	6
3.REQUISITOS DE SISTEMA	8
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE	8
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE	10
4. DIAGRAMAS DE UML.....	11
4.1 CASOS DE USO.....	11
4.2 DIAGRAMA DE CLASSE	11
5. Especificação estrutura de uma rotina do sistema.....	11
5.1 – Apresentar a especificação de acordo com o modelo abaixo.....	11
5.2 – Apresentar a codificação da rotina	12
6. ARQUITETURA DO SISTEMA	12
6.1. Visão Geral do Sistema.....	12
6.2. Arquitetura Geral	12
6.3. Componentes Arquiteturais e Camadas	13
6.4. Diagrama da Arquitetura.....	13
6.5. Organização de Pastas e Módulos	14
6.6. Integrações Externas (se houver)	15
6.7. Ambiente e Implantação	15
8. REFERÊNCIAS BIBLIOGRÁFICAS.....	15

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software



1.2 Apresentação

A Arcor (<https://www.arcor.com.br/>) atua no ramo alimentício com um portfólio que reúne chocolates, biscoitos, guloseimas e outras categorias. Como ação de marketing e relacionamento com o público, a empresa propõe às equipes do 1º semestre do curso de Análise e Desenvolvimento de Sistemas o desenvolvimento de um sistema digital com caráter lúdico e/ou educativo, capaz de aproximar o jogador da marca Arcor e de suas diferentes marcas, linhas e produtos.

A proposta central é transformar o conhecimento sobre o portfólio em uma experiência de jogo. O participante deverá avançar por perguntas, desafios, pistas e feedbacks que apresentem marcas como Tortugueta e Triunfo, além de outras marcas e produtos selecionados pela equipe em conjunto com o parceiro. O conteúdo utilizado deverá ser baseado em materiais oficiais da Arcor e validado antes da entrega final, evitando afirmações promocionais, nutricionais ou de qualidade que não estejam apoiadas em fontes fornecidas ou aprovadas pela empresa.

Cada equipe poderá adotar uma das duas abordagens abaixo ou combinar as duas em uma única solução:

- **Modo Quiz:** sequência de perguntas de múltipla escolha, verdadeiro/falso ou associação, com pontuação, níveis de dificuldade, feedback imediato e informações complementares sobre a marca ou produto relacionado.
- **Modo Descoberta:** mecânica de adivinhação por perguntas sucessivas, inspirada no conceito de jogos como o Akinator, em que o sistema utiliza respostas do jogador para restringir possibilidades e tentar identificar uma marca, linha ou produto. A referência deve ser apenas conceitual, sem copiar identidade visual, conteúdo, código ou elementos proprietários do serviço citado.

Requisito obrigatório do projeto: a base de perguntas, alternativas, respostas, pistas e respectivos vínculos com marcas/produtos deverão estar armazenada em um Banco de Dados relacional. Não será aceita como solução final uma aplicação cuja base de perguntas esteja integralmente codificada diretamente no programa.

A equipe deverá definir e justificar o público-alvo da experiência (por exemplo, adolescentes, jovens adultos ou público geral) e adequar linguagem, dificuldade, duração das partidas e interface ao perfil escolhido.

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Criação de um quiz que envolva o jogador ou jogadores com as seguintes marcas: Bon o Bon e Tortuguita. O jogo trará um quiz contra a máquina, onde será o jogador contra a concorrência, cada um com uma barra de vida que responderá 7 perguntas. O objetivo é **eliminar a concorrência**. O segundo modo é uma luta entre 2 jogadores, onde eles decidirão qual marca é a **preferida da galera**. Haverá o nível iniciante e “Estupido” (Modo hard).

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

// Perguntas com imagem (dependendo da pergunta), perguntas com seleção – Caio e Pedro

// mascote Bon o Bon: BON o BONbado – design com chocolate com músculos e sapato (ideia Matheus)

15 perguntas de cada marca

2.2.2 – Pesquisa Desk

Mapeamento de quizzes famosos e análise do que chama atenção

Mapeamento de concorrentes da Tortuguita e Bon o Bon

Artigos sobre quiz

Site modelos, como o genially.com/pt-br/modelos/quis/

Pesquisas nas redes sociais das marcas selecionadas

2.2.3 – Como podemos? How Might We (HMW)?

Como podemos preencher o banco de dados?

Tabela: Perguntas, Respostas, Imagem Perguntas, Reposta Imagem, Ranking, Nome Jogador, Progresso.

Como podemos linkar o Gamefy no programa?

Via QR code

Como podemos linkar o banco de dados no servidor?

Será realizado, neste primeiro momento via local.

Como podemos deixar mais atrativo e divertido?

Design chamativo, chocolate caso gabarite o quis, perguntas interessantes.

Como podemos incluir bonificação de pontos para melhorar a pontuação?
Multiplicador de acertos com base no número de perguntas

2.3 Ideação – desenho da solução (trilha do usuário)

1- “E se...?”

Se você tivesse super poderes?

Caio: Poder de criar um Super Servidor para o jogo

Matheus: Super velocidade para fazer o software em 5 min

Pedro: Super atenção para não deixar nenhum detalhe passar

Guilherme: Super inteligência para programar o melhor software

Se você tivesse muito dinheiro?

Caio: Investiria em um super servidor para o software

Matheus: Investiria em design

Pedro: Investiria em marketing

Guilherme: Investiria em uma mega estrutura para o software, para ter escalabilidade do programa

Se você tivesse nenhum dinheiro?

Todos buscariam o conhecimento para realizar o software de formas alternativas.

E se fosse o Dick Vigarista e quisesse boicotar?

Todos sabotariam os outros trabalhos, a fim de receber todos os bônus.

2- Crazy 8s

Chocolate grátis para cada questão correta.

Questões infinitas

Premiação para o jogador com mais acertos

Modo speedrun

Placar de líderes mundiais

Cartão vitalício da Arcor para o jogador com mais acertos

Visita à Arcor para os três primeiros colocados

Possibilidade da criação de um novo produto relacionado ao software

3- Trilha do usuário

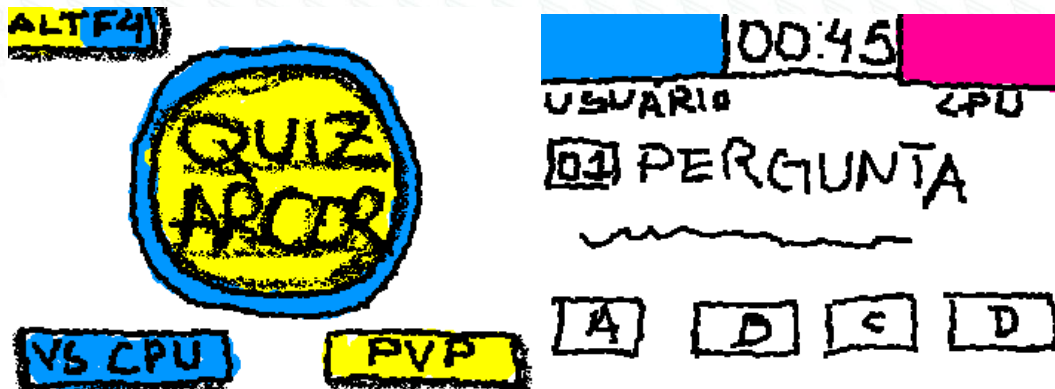
Exemplo 1 – Jogo quiz sobre marcas da Arcor

Usuário: José, estudante do ensino médio. Objetivo: Interagir com o quiz até o final.

Etapa	Descobrir	Escolher	Responder	Resultado
O que o jogador faz	Abrir o app	Escolher o modo do jogo	Ler a pergunta e responder o quiz	Vê o seu desempenho
Como ele se sente	Curioso	Animado	Inseguro	Satisfeito ou insatisfeito com o resultado
Onde ele trava	Escolher nome de usuário	Decidir entre um dos modos	Na indecisão de responder	Se o resultado for não esperado



2.4 Prototipagem



3. REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RF01	
Função	Tela inicial e instruções
Descrição	Apresentar identidade do projeto, objetivo do jogo, instruções de uso e opção para iniciar uma partida.
Entradas	Logotipo Botão Iniciar Escolher modo Opção de instruções
Fonte	VS Code C#
Saídas	Redirecionamento para a tela escolhida pelo usuário
Ação	Botão iniciar vai para a tela de escolher modo Em escolher modo tem duas opções, jogo contra a máquina ou contra um amigo, ao escolher uma, vai para tela de Identificação de jogador Opção de instruções vai para outra tela de informações passo a passo de como usar o aplicativo Botão de voltar a tela inicial

RF02	
Função	Identificação do jogador
Descrição	Permitir entrada de nickname/apelido ou início anônimo sem exigir dados pessoais desnecessários.
Entradas	Nickname/apelido Confirmar Confirmar em modo anônimo
Fonte	VS Code

	C#
Saídas	Confirmar nickname? Confirmar Anonimato?
Ação	Barra de escrita onde o usuário preencherá com seu apelido Botão de confirmar nickname continuará ao quiz com o apelido escolhido pelo usuário Botão de confirmar anonimato continuará ao quiz sem apelido escolhido, apenas como jogador(a) 1 ou 2
RF03	
Função	Carga de conteúdo pelo Banco de Dados
Descrição	Recuperar perguntas, alternativas, pistas, marcas/produtos e parâmetros de jogo diretamente do Banco de Dados relacional.
Entradas	
Fonte	
Saídas	
Ação	

RF04	
Função	Seleção do modo de jogo
Descrição	Iniciar pelo menos uma mecânica principal: Quiz ou Descoberta por perguntas sucessivas. A combinação das duas poderá ser tratada como evolução.
Entradas	Modo de jogo VS CPU PVP
Fonte	VS Code C#
Saídas	Confirmar modo?
Ação	Após o usuário ter selecionado a opção escolher modo, ele é direcionado para a tela com as duas opções Escolhendo o modo VS CPU, ele irá jogar contra a máquina Escolhendo o modo PVP, ele irá jogar contra um amigo ao lado dele
RF05	
Função	Execução do Quiz
Descrição	Quando adotado, apresentar perguntas e alternativas, registrar a resposta, validar o resultado e avançar para a próxima questão.
Entradas	

Fonte	
Saídas	
Ação	

RF06	
Função	Execução do modo Descoberta
Descrição	Quando adotado, coletar respostas sucessivas e aplicar regras lógicas para reduzir possibilidades até sugerir uma marca, linha ou produto
Entradas	
Fonte	
Saídas	
Ação	

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	
Descrição	
Entradas	
Fonte	
Saídas	
Ação	

RFS02	
Função	
Descrição	
Entradas	

Fonte	
Saídas	
Ação	

4. DIAGRAMAS DE UML

4.1 CASOS DE USO

Apresentar 3 casos de uso do sistema

4.2 DIAGRAMA DE CLASSE

Apresentar 1 diagrama de classe do sistema

5. Especificação estrutura de uma rotina do sistema

5.1 – Apresentar a especificação de acordo com o modelo abaixo

Sistema/Software

Função

Descrição

Entradas

Fonte

Saídas

Destino

Ação

•

Requisitos

Pré-condição

Pós-condições

Efeitos colaterais

5.2 – Apresentar a codificação da rotina

Apresente em pseudocódigo, Python ou outra linguagem

6. ARQUITETURA DO SISTEMA

Para a arquitetura do sistema, considere o template abaixo:

Template (esquema de arquitetura) para orientar a elaboração de um **Projeto de Arquitetura de Sistema Web**, cobrindo desde a camada de apresentação até a persistência de dados, com foco em modularidade, escalabilidade e eficiência.

6.1. Visão Geral do Sistema

- **Nome do Sistema:**
- **Objetivo do Sistema:** (Breve descrição da finalidade do sistema)
- **Público-Alvo:**
- **Principais Funcionalidades:**

6.2. Arquitetura Geral

Descrever o estilo arquitetural adotado (por exemplo, MVC, camadas, microsserviços, hexagonal, etc.):

Exemplo: Arquitetura em Camadas (Layered Architecture) com separação clara entre apresentação, lógica de negócios e persistência.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias:** (Ex.: HTML5, CSS3, JavaScript, React/Vue/Angular)
- **Responsabilidades:**
 - Interface com o usuário
 - Validação inicial de dados
 - Comunicação com a API via HTTP/JSON

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias:** (Ex.: Node.js, Django, Spring Boot, Flask, Laravel)
- **Responsabilidades:**
 - Implementação da lógica de negócio
 - Autenticação/autorização
 - Comunicação com a camada de dados
 - Exposição de endpoints RESTful/GraphQL

6.3.3. Camada de Domínio (Negócio)

- **Descrição:**
 - Contém as regras de negócio e os modelos de domínio
 - Independente das demais camadas
- **Componentes:**
 - Entidades
 - Casos de uso
 - Regras de validação

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias:** (Ex.: PostgreSQL, MySQL, MongoDB, SQLite)
- **Responsabilidades:**
 - Armazenamento e recuperação de dados
 - Scripts de criação e migração de banco

6.4. Diagrama da Arquitetura

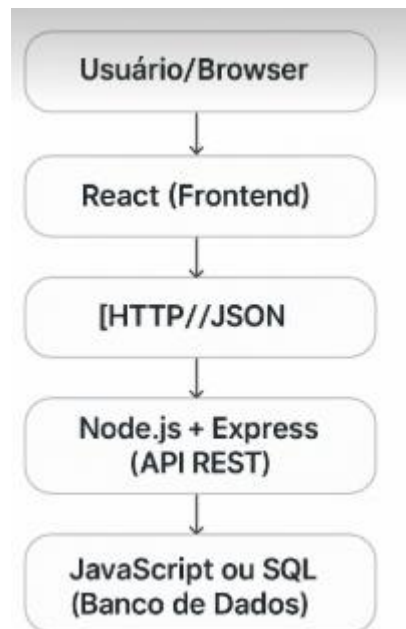
Inserir um **diagrama estrutural** (ex.: em draw.io, Lucidchart ou outro) mostrando:

- Fluxo entre frontend, backend, banco de dados
- Componentes principais
- APIs e serviços envolvidos
- Gateways, autenticação, filas de mensagens (se houver)

Exemplo textual de uma arquitetura de sistema web.

```

[Usuário/Browser]
|
React (Frontend)
|
[HTTP/JSON]
|
Node.js + Express (API REST)
|
JavaScript ou SQL
|
PostgreSQL (Banco de Dados)
Diagrama:
    
```



6.5. Organização de Pastas e Módulos

Descrever como o código será organizado no repositório:

```

├── /frontend
│   ├── components/
│   ├── pages/
│   └── services/
├── /backend
│   ├── controllers/
│   ├── services/
│   ├── models/
│   ├── routes/
│   └── config/
    
```

```
└─ /database
   └─ schema.sql
   └─ migrations/
```

6.6. Integrações Externas (se houver)

- APIs de terceiros (pagamentos, mapas, redes sociais)
- Serviços de autenticação (OAuth2, Firebase, etc.)
- Serviços de envio (email, notificações)

6.7. Ambiente e Implantação

- **Ambientes:** desenvolvimento, homologação, produção
- **Contêineres:** Docker/Docker Compose (se usado)
- **CI/CD:** GitHub Actions, GitLab CI, Jenkins
- **Hospedagem:** Heroku, Vercel, AWS, GCP, Azure

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software**. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.